

MỤC LỤC

MỤC LỤC	1
Phần 1: Database (Tập trung vào CSDL quan hệ)	2
1. Cài đặt DBMS.....	2
2. Các câu lệnh cơ bản với SQL.....	2
Phần 2: OOP	9
3. OOP trong Java.....	9
a, Biến (Variables).....	9
b. Hàm/Phương thức (Methods).....	9
c. Nhập/Xuất Cơ bản (I/O - Input/Output).....	9
d. Vòng lặp (Loops).....	10
e. Thao tác File Cơ bản (File I/O).....	10
f. Class (Lớp).....	11
g. Object (Đối tượng).....	11
h. Abstract Class (Lớp Trừu Tượng).....	12
i. Interface (Giao diện).....	13
4. Dependency Injection (DI), Inversion of Control (IoC).....	14
a. Inversion of Control (IoC) - Đảo ngược Điều khiển.....	14
b. Dependency Injection (DI) - Tiêm Phụ thuộc.....	14
c. Ứng dụng DI và IoC trong Spring.....	14

Phần 1: Database (Tập trung vào CSDL quan hệ)

1. Cài đặt DBMS

- Cài đặt MySQL

2. Các câu lệnh cơ bản với SQL

Bài toán: Quản lý thư viện.

- Định nghĩa Dữ liệu (DDL): CREATE TABLE, DROP TABLE, ALTER TABLE, CREATE VIEW (tham khảo thêm), DROP VIEW (tham khảo thêm)

+ Tạo bảng:

```
1 • USE typ_tuan_1_db;
2 • CREATE TABLE NhanVien (
3     MaNV INT PRIMARY KEY,
4     TenNV VARCHAR(50),
5     ChucVu VARCHAR(50),
6     Luong DECIMAL(10,2)
7 );
8
9 • CREATE TABLE Sach (
10     MaSach INT PRIMARY KEY,
11     TenSach VARCHAR(255),
12     TacGia VARCHAR(225),
13     NamXB INT NOT NULL
14 );
15
```

+ Xóa bảng (xóa cấu trúc bảng kèm dữ liệu của nó):

```
1 • USE typ_tuan_1_db;
2 • DROP TABLE Sach;
3
```

+ ALTER: Thay đổi cấu trúc bảng:

```
1 • USE typ_tuan_1_db;
2 • ALTER TABLE Sach
3     ADD COLUMN Gia INT NOT NULL
```

Kết hợp với ADD COLUMN

Kết hợp DROP COLUMN

Thay đổi kiểu dữ liệu, kích thước hoặc ràng buộc của một cột đã có.

```
1 • USE typ_tuan_1_db;
2 • ALTER TABLE Sach
3   MODIFY COLUMN TenSach VARCHAR(300) NOT NULL;
```

Thêm/Xóa Ràng Buộc (ADD/DROP CONSTRAINT)

- Thao tác Dữ liệu (DML): SELECT, INSERT, UPDATE, DELETE

```
1 • USE typ_tuan_1_db;
2
3 • INSERT INTO NhanVien (MaNV, TenNV, ChucVu, Luong)
4   VALUE (001, "Nguyen Van Hoang", "Giam Doc", 10000.00);
5
```

```
1
2 • SELECT * FROM nhanvien;
3
```

```
4 • SELECT TenNV, ChucVu
5   FROM NhanVien
6  WHERE Luong > 20000;
```

```
9 • UPDATE NhanVien
10  SET Luong = 200000.00
11  WHERE MaNV = 1;
```

```
13 • DELETE FROM NhanVien
14  WHERE MaNV = 101;
```

- Trigger, Transaction: CREATE PROCEDURE, CREATE FUNCTION, CREATE TRIGGER, COMMIT, ROLLBACK, SAVE TRANSACTION

```
1 • DROP PROCEDURE IF EXISTS ThemSachMoi;
2
3 DELIMITER //
4 • CREATE PROCEDURE ThemSachMoi (
5     IN p_MaSach INT,
6     IN p_TenSach VARCHAR(255),
7     IN p_TacGia VARCHAR(225),
8     IN p_Gia INT,
9     IN p_NamXB INT
10 )
11 BEGIN
12     INSERT INTO Sach (MaSach, TenSach, TacGia, Gia, NamXB)
13     VALUES (p_MaSach, p_TenSach, p_TacGia, p_Gia, p_NamXB);
14 END //
15 DELIMITER ;
16
17 • CALL ThemSachMoi(10, 'Sapiens', 'Yuval Noah Harari', 180000, 2011);
18
```

```

1 • DROP FUNCTION IF EXISTS TinhTienThuong;
2
3 DELIMITER //
4 CREATE FUNCTION TinhTienThuong (
5     p_Luong DECIMAL(10, 2),
6     p_TyLe DECIMAL(3, 2)
7 )
8 RETURNS DECIMAL(10, 2)
9 DETERMINISTIC
10 BEGIN
11     DECLARE v_TienThuong DECIMAL(10, 2);
12     SET v_TienThuong = p_Luong * p_TyLe;
13     RETURN v_TienThuong;
14 END //
15 DELIMITER ;

```

```

1 • SELECT
2     MaNV,
3     TenNV,
4     ChucVu,
5     Luong,
6     TinhTienThuong(Luong, 0.15) AS TienThuong15PhanTram,
7     (Luong + TinhTienThuong(Luong, 0.15)) AS TongThuNhap
8 FROM
9     NhanVien;

```

Result Grid						
Filter Rows:						
Export:  Wrap Cell Content: 						
	MaNV	TenNV	ChucVu	Luong	TienThuong15PhanTram	TongThuNhap
▶	1	Nguyen Van Hoang	Giam Doc	200000.00	30000.00	230000.00
	2	Nguyen Van Huy	Pho Giam Doc	5000.00	750.00	5750.00

```
1 • DROP TRIGGER IF EXISTS trg_TangLuongKhiThemSach;  
2  
3 DELIMITER //  
4 • CREATE TRIGGER trg_TangLuongKhiThemSach  
5 AFTER INSERT ON Sach  
6 FOR EACH ROW  
7 BEGIN  
8     UPDATE NhanVien  
9     SET Luong = Luong + 10000.00  
10    WHERE MaNV = 1;  
11  
12 END //  
13 DELIMITER ;
```

```
1 • START TRANSACTION;  
2  
3 • UPDATE NhanVien  
4 SET Luong = Luong + 5000000.00  
5 WHERE MaNV = 2;  
6  
7 • UPDATE Sach  
8 SET Gia = Gia * 1.05  
9 WHERE MaSach = 1;  
10  
11 • COMMIT;
```

```

1 • START TRANSACTION;
2 • UPDATE NhanVien
3   SET Luong = 4000000.00
4   WHERE MaNV = 2;
5
6   -- SELECT * FROM NhanVien WHERE MaNV = 2;
7 • ROLLBACK;
8 • START TRANSACTION;
9
10 • UPDATE NhanVien
11   SET Luong = 40000000.00 -- Cập nhật đúng
12   WHERE MaNV = 2;
13 • COMMIT;

```

```

1 • START TRANSACTION;
2
3 • INSERT INTO Sach (MaSach, TenSach, TacGia, Gia, NamXB)
4   VALUES (101, 'Lược Sử Thời Gian', 'Stephen Hawking', 150000, 1988);
5
6 • SAVEPOINT ThemMotSach;
7
8   -- Giả sử MaNV = 1 lỗi khóa chính (Primary Key)
9 • INSERT INTO NhanVien (MaNV, TenNV, ChucVu, Luong)
10  VALUES (1, 'Nguyễn Văn C', 'Bảo vệ', 8000000.00);
11  -- Lỗi!
12
13 • ROLLBACK TO SAVEPOINT ThemMotSach;
14
15 • INSERT INTO NhanVien (MaNV, TenNV, ChucVu, Luong)
16  VALUES (104, 'Nguyễn Văn D', 'Bảo vệ', 8000000.00);
17
18 • COMMIT;

```

- Điều khiển Truy cập: GRANT, REVOKE

```

20 • GRANT SELECT
21   ON Sach
22   TO 'thu_kho'@'%';
23
24 • GRANT SELECT, INSERT, UPDATE, DELETE
25   ON NhanVien
26   TO 'quan_ly'@'localhost'; -- chỉ truy cập local
27
28 • REVOKE SELECT
29   ON Sach
30   FROM 'thu_kho'@'%';
31
32 • REVOKE UPDATE
33   ON NhanVien
34   FROM 'quan_ly'@'localhost';

```

- Toán tử truy vấn: FROM, WHERE, ORDER BY, HAVING, AND, OR,...

```

1 • SELECT
2   ChucVu,
3   COUNT(MaNV) AS SoLuongNhanVien,
4   AVG(Luong) AS LuongTrungBinh
5 FROM
6   NhanVien
7 WHERE
8   Luong > 10000000
9   AND
10  (ChucVu = 'Quản lý' OR ChucVu = 'Thủ thư') --
11 GROUP BY
12   ChucVu
13 HAVING
14   AVG(Luong) BETWEEN 15000000 AND 30000000 -- 4. HAVING: Lọc
15 ORDER BY
16   LuongTrungBinh DESC;
17   -- 5. ORDER BY: Sắp xếp kết quả theo Lương Trung Bình giảm dần
18

```


Phần 2: OOP

3. OOP trong Java

Java cơ bản (object, class, abstract class, interface, biến, hàm, I/O, vòng lặp, thao tác file)

a, Biến (Variables)

Biến là nơi lưu trữ dữ liệu. Trong Java, biến phải được khai báo kiểu dữ liệu và tên trước khi sử dụng.



```
// Khai báo biến
int soLuong;
String tenSach;
boolean daMuon;

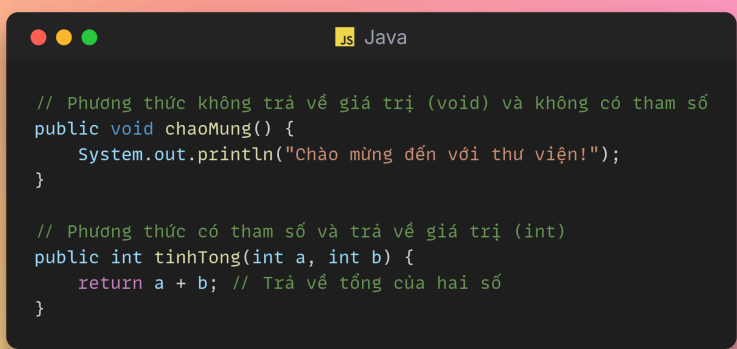
// Khởi tạo (gán giá trị)
soLuong = 5;
tenSach = "Java Core";
daMuon = true;

// Khai báo và khởi tạo cùng lúc
double giaSach = 150.50;
char chucai = 'A';
```

Hoang

b. Hàm/Phương thức (Methods)

Hàm (hay Phương thức trong OOP) là một khối code thực hiện một tác vụ cụ thể. Nó giúp tái sử dụng logic và làm cho code có cấu trúc hơn.



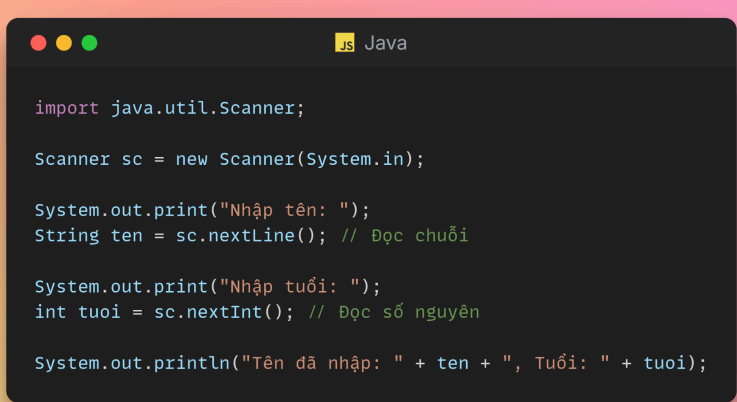
```
// Phương thức không trả về giá trị (void) và không có tham số
public void chaoMung() {
    System.out.println("Chào mừng đến với thư viện!");
}

// Phương thức có tham số và trả về giá trị (int)
public int tinhTong(int a, int b) {
    return a + b; // Trả về tổng của hai số
}
```

Hoang

c. Nhập/Xuất Cơ bản (I/O - Input/Output)

I/O cơ bản dùng để giao tiếp với người dùng qua Console (cửa sổ dòng lệnh).



```
import java.util.Scanner;

Scanner sc = new Scanner(System.in);

System.out.print("Nhập tên: ");
String ten = sc.nextLine(); // Đọc chuỗi

System.out.print("Nhập tuổi: ");
int tuoi = sc.nextInt(); // Đọc số nguyên

System.out.println("Tên đã nhập: " + ten + ", Tuổi: " + tuoi);
```

Hoang

d. Vòng lặp (Loops)

Vòng lặp dùng để thực thi một khối code nhiều lần.

```
for (int i = 1; i ≤ 5; i++) {
    System.out.println("Lần lặp thứ: " + i);
}

int dem = 0;
while (dem < 3) {
    System.out.println("Vẫn đang lặp...");
    dem++;
}
```

Hoang

e. Thao tác File Cơ bản (File I/O)

Thao tác với File dùng để đọc hoặc ghi dữ liệu vào các tệp trên hệ thống.

```
import java.io.FileWriter;
import java.io.IOException;

try {
    FileWriter writer = new FileWriter("du_lieu.txt");
    writer.write("Đây là dòng dữ liệu cần ghi.");
    writer.close(); // Đóng writer
    System.out.println("Đã ghi file thành công.");
} catch (IOException e) {
    e.printStackTrace();
}
```

Hoang

```
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

try {
    File file = new File("du_lieu.txt");
    Scanner fileReader = new Scanner(file);

    while (fileReader.hasNextLine()) {
        String data = fileReader.nextLine();
        System.out.println("Đọc từ file: " + data);
    }
    fileReader.close();
} catch (FileNotFoundException e) {
    System.out.println("Không tìm thấy file.");
}
```

Hoang

f. Class (Lớp)

Class là một khuôn mẫu hoặc bản thiết kế để tạo ra các đối tượng (objects). Nó định nghĩa các thuộc tính và hành vi mà mọi đối tượng được tạo ra từ lớp đó sẽ có.



```

public class Sach {
    // Thuộc tính
    String tenSach;
    String tacGia;
    int namXuatBan;

    // Phương thức (Hành vi)
    public void hienThiThongTin() {
        System.out.println(tenSach + tacGia);
    }
}

```

Hoang

g. Object (Đối tượng)

Object là một thực thể cụ thể được tạo ra từ một Class. Mỗi Object là một thể hiện (instance) của Class và lưu trữ dữ liệu thực tế tại thời điểm chạy chương trình.



```

// Tạo đối tượng 'sach1' từ Class Sach
Sach sach1 = new Sach();

// Gán giá trị cho thuộc tính của đối tượng
sach1.tenSach = "Đắc Nhân Tâm";
sach1.tacGia = "Dale Carnegie";
sach1.namXuatBan = 1936;

// Gọi phương thức của đối tượng
sach1.hienThiThongTin();
// Kết quả: Tên: Đắc Nhân Tâm, Tác giả: Dale Carnegie

```

Hoang

h. Abstract Class (Lớp Trừu Tượng)

Abstract Class là một Class không thể tạo ra Object trực tiếp. Nó được dùng để cung cấp một khuôn khổ chung cho các lớp con kế thừa. Abstract Class có thể chứa cả phương thức đã được định nghĩa hoàn chỉnh và phương thức trừu tượng (abstract method) (phương thức không có phần thân, bắt buộc lớp con phải triển khai).



```
public abstract class NhanVien {
    String hoTen;
    int luongCoBan = 1000;

    // Phương thức hoàn chỉnh
    public void hienThiHoTen() {
        System.out.println("Họ tên: " + hoTen);
    }

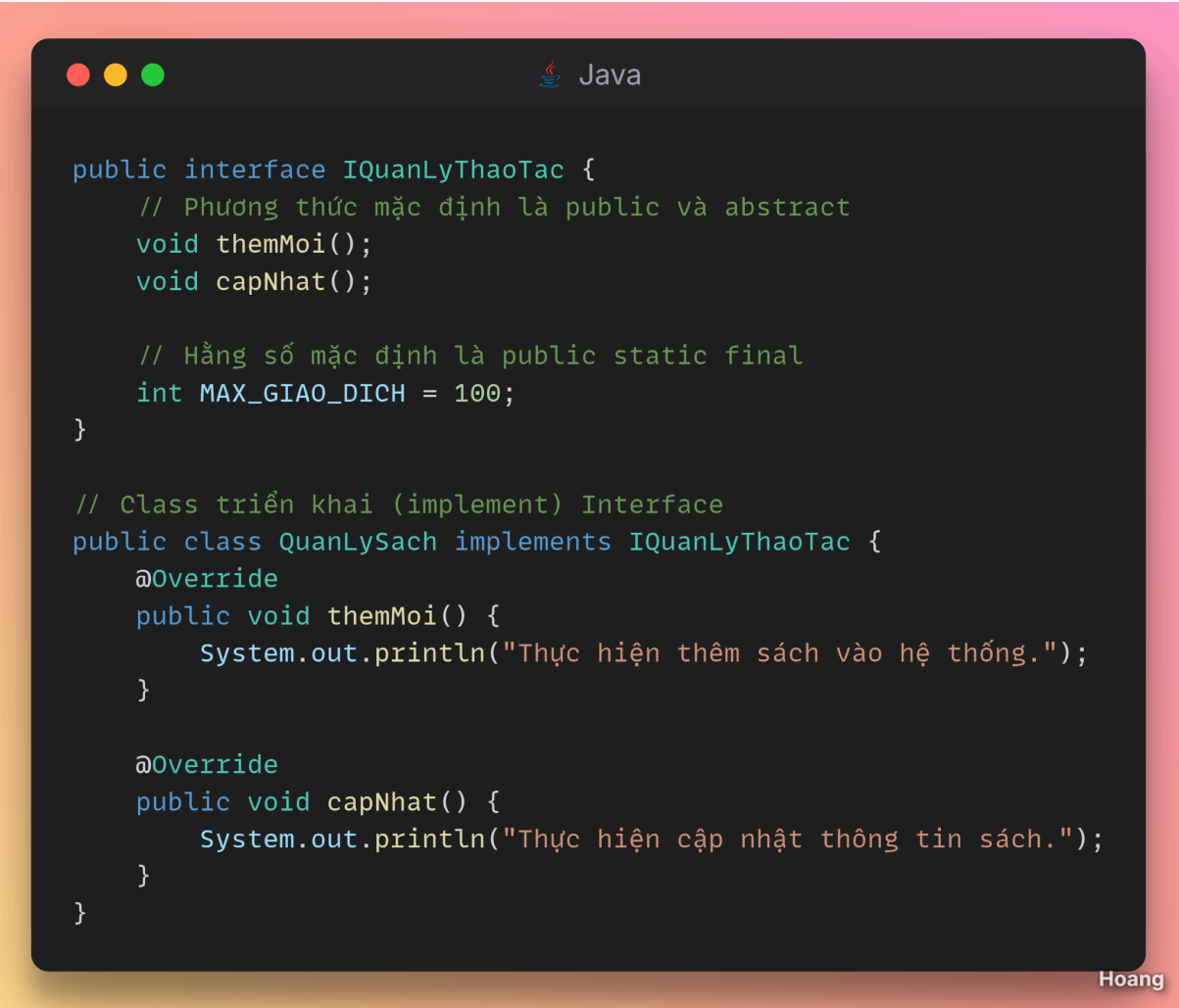
    // Phương thức trừu tượng (bắt buộc lớp con phải định nghĩa)
    public abstract double tinhLuongThucLanh();
}

// Lớp con phải triển khai phương thức tinhLuongThucLanh()
public class ThuThu extends NhanVien {
    @Override
    public double tinhLuongThucLanh() {
        return luongCoBan * 1.5;
        // Định nghĩa cụ thể cho lớp ThuThu
    }
}
```

Hoang

i. Interface (Giao diện)

Interface là một loại hợp đồng trong Java, chỉ chứa các khai báo về các phương thức công cộng và trừu tượng (public abstract methods) (từ Java 8 có thể có thêm default method và static method). Interface định nghĩa một tập hợp các hành vi mà bất kỳ Class nào muốn triển khai (implement) nó đều phải tuân theo.



```
public interface IQuanLyThaoTac {  
    // Phương thức mặc định là public và abstract  
    void themMoi();  
    void capNhat();  
  
    // Hằng số mặc định là public static final  
    int MAX_GIAO_DICH = 100;  
}  
  
// Class triển khai (implement) Interface  
public class QuanLySach implements IQuanLyThaoTac {  
    @Override  
    public void themMoi() {  
        System.out.println("Thực hiện thêm sách vào hệ thống.");  
    }  
  
    @Override  
    public void capNhat() {  
        System.out.println("Thực hiện cập nhật thông tin sách.");  
    }  
}
```

Hoang

4. Dependency Injection (DI), Inversion of Control (IoC)

a. Inversion of Control (IoC) - Đảo ngược Điều khiển

Inversion of Control (IoC) là một nguyên tắc thiết kế trong lập trình hướng đối tượng. Nó đảo ngược luồng điều khiển truyền thống của chương trình:

- Truyền thông: Các lớp tự tạo ra các đối tượng mà chúng cần, hoặc tự tìm kiếm chúng.
- IoC: Một framework (hoặc container) bên ngoài chịu trách nhiệm tạo ra và quản lý vòng đời của các đối tượng, sau đó *tiêm* (inject) chúng vào nơi cần dùng.

b. Dependency Injection (DI) - Tiêm Phụ thuộc

Dependency Injection (DI) là một mẫu thiết kế (Design Pattern) cụ thể để thực hiện nguyên tắc IoC. DI là quá trình một đối tượng được cung cấp các phụ thuộc (các đối tượng khác mà nó cần để hoạt động) bởi một bên thứ ba (thường là IoC Container), thay vì tự tạo ra chúng.

- Phụ thuộc (Dependency): Là một đối tượng mà một đối tượng khác cần để thực hiện công việc của mình. Ví dụ, một lớp `QuanLySach` cần một lớp `DatabaseService` để lưu trữ dữ liệu.

DI giúp đạt được sự ghép nối lỏng lẻo (Loose Coupling), làm cho code dễ kiểm thử, tái sử dụng và mở rộng hơn.

c. Ứng dụng DI và IoC trong Spring

Spring Framework được xây dựng hoàn toàn dựa trên nguyên tắc IoC. Nó sử dụng mô hình DI để quản lý các thành phần của ứng dụng, giúp code sạch hơn, linh hoạt hơn và dễ kiểm thử hơn.

Trái tim của Spring là IoC Container. Đây là một chương trình chịu trách nhiệm:

- Tạo đối tượng (Instantiation): Tạo các đối tượng của bạn (được gọi là Beans trong Spring).
- Quản lý vòng đời (Lifecycle Management): Quản lý Bean từ khi khởi tạo đến khi bị hủy.

- **Tiêm phụ thuộc (Dependency Injection):** Tự động phát hiện và cung cấp (tiêm) các Bean cần thiết cho nhau.

Nói cách khác, IoC Container giữ quyền kiểm soát đối với việc tạo và quản lý đối tượng.

Nhờ IoC và DI, lập trình viên trong Spring không cần viết bất kỳ dòng code nào như `new ProductService()` hay `new CoffeeMachine()`. Spring Container làm tất cả công việc đó, giúp:

1. **Ghép nối Lỏng lẻo:** Dễ dàng thay thế các thành phần dịch vụ mà không cần sửa đổi lớp sử dụng.
2. **Dễ Kiểm thử (Testable):** Có thể dễ dàng tiêm các đối tượng giả (Mock Objects) vào để kiểm thử các thành phần một cách độc lập.
3. **Tái sử dụng:** Các Bean có thể được tái sử dụng khắp nơi trong ứng dụng.

